

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

36 - Control Unit

Introduction

- We look at possible implementation of MIPS subset
- Build this implementation using datapath and adding control function
- Implementation covers `lw`, `sw`, `beq`, and arithmetic-logical instructions `add`, `sub`, `AND`, `OR`, and `slt`
- Enhance design to include `j` instruction

ALU Control

- MIPS ALU defines the 6 following combinations of four control inputs

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

ALU Control

- Depending on instruction class, ALU will need to perform one of these first five functions
 - *lw* and *sw* use ALU to compute memory address by **addition**
 - R-type, ALU needs to perform one of five actions (**AND**, **OR**, **subtract**, **add**, or **slt**), depending on value of 6-bit *funct* field
 - *beq*, ALU must perform **subtraction**

ALUOp Control Bits

- Generate 4-bit ALU control input using small control unit that has as inputs
 - *funct* field of instruction and
 - 2-bit control field, called **ALUOp**
- **ALUOp** indicates whether operation to be performed should be
 - **add** (**00**) for loads and stores
 - **subtract** (**01**) for beq, or
 - **operation** encoded in *funct* field (**10**)

ALU Control Bits

- Output of ALU control unit is 4-bit signal that directly controls ALU by generating one of 4-bit combinations shown previously
- Table shows how to set ALU control inputs based on
 - 2-bit *ALUOp* control and
 - 6-bit *function* code
- Later, we will see how **ALUOp** bits are generated from the main control unit

ALU Control Bits

Instruction opcode	ALUOp	Instruction operation	Funct field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	0010
SW	00	store word	XXXXXX	add	0010
Branch equal	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
R-type	10	subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	set on less than	101010	set on less than	0111

Multiple Level Control

- Style of using multiple levels of decoding, main control unit generates ALUOp bits, which then are used as input to ALU control that generates actual signals to control the ALU unit, is a common implementation technique
 - Using multiple levels of control can reduce size of main control unit
 - Using several smaller control units may increase speed of control unit
- Such optimizations are important, since speed of control unit is often critical to clock cycle time

ALUOp Control Bits

- There are several different ways to implement mapping from 2-bit *ALUOp* and 6-bit *funct* field to 4 ALU operation control bits
- Because only small number of 64 possible values of *funct* field are of interest and *funct* field is used only when *ALUOp* bits equal 10, we can use small piece of logic that recognizes subset of possible values and causes correct setting of ALU control bits

ALUOp Control Bits

- As a step in designing this logic, it is useful to create truth table for the interesting combinations of function code field and ALUOp bits
- This truth table shows how 4-bit ALU control is set depending on these two input fields

ALUOp Control Bits

ALUOp		Funct field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

ALUOp Control Bits

- Since full truth table is very large ($2^8 = 256$ entries) and we don't care about value of ALU control for many of these input combinations, we show only truth table entries for which ALU control must have a specific value
- We will use this practice of showing only truth table entries for outputs that must be asserted and not showing those that are all deasserted or don't care

ALUOp Control Bits

- Because in many instances we do not care about values of some of **inputs**, and because we wish to keep the tables compact, we also include **don't-care terms**
- A don't-care term in truth table (X) indicates that output does not depend on value of input corresponding to that column
 - For example, when ALUOp bits are 00, as in the first row of figure, we always set ALU control to 0010, independent of function code

ALUOp Control Bits

- In this case, *function* code inputs will be don't cares in this line of truth table
- Later, we will see examples of another type of don't-care term
- Once truth table has been constructed, it can be optimized and then turned into gates

Designing Main Control Unit

- Now that we have described how to design ALU that uses the function code and 2-bit signal as its control inputs, we can return to looking at the rest of control
- To start this process, let's identify fields of instruction and control lines that are needed for the datapath we constructed
- To understand how to connect fields of instruction to datapath, it is useful to review the formats of the three instruction classes: R-type, branch, and load-store instructions

Instruction Formats

Field	0	rs	rt	rd	shamt	funct
Bit positions	31:26	25:21	20:16	15:11	10:6	5:0

a. R-type instruction

Field	35 or 43	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

b. Load or store instruction

Field	4	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

c. Branch instruction

Instruction Formats

- Several major observations about this instruction format
 - *op* field is always contained in bits 31:26; Op[5:0]
 - Two registers to be read are always specified by *rs* and *rt* fields, at bit positions 25:21 and 20:16
 - True for R-type instructions, branch equal, and store
 - Base register for *load* and *store* is always in bits 25:21 (*rs*)
 - 16-bit offset for *branch equal*, *load*, and *store* is always in bits 15:0
 - Destination register is in one of two places
 - For load it is in bit positions 20:16 (*rt*)
 - For R-type instruction it is in bit positions 15:11 (*rd*)

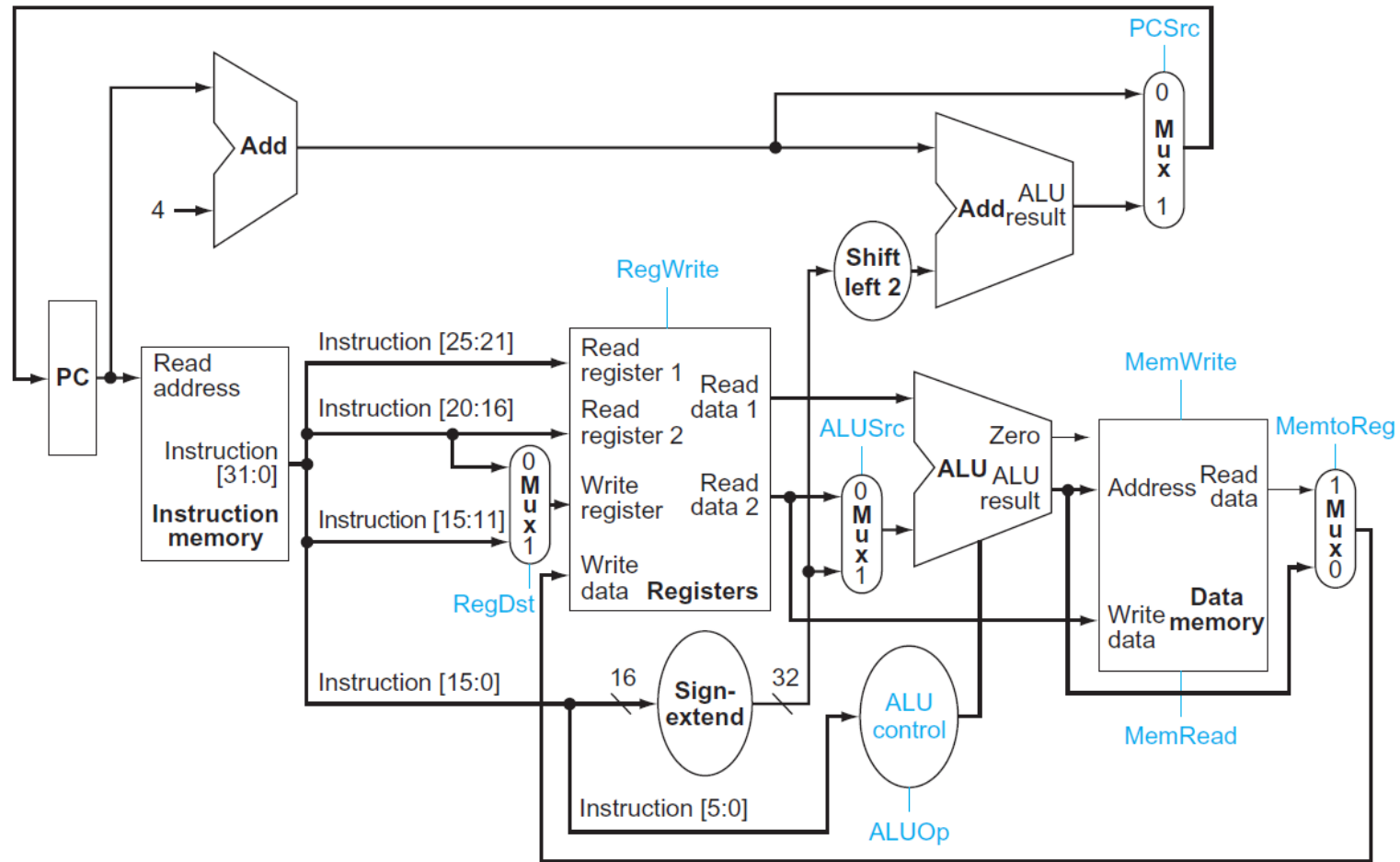
Instruction Formats

- Thus, we will need to add multiplexor to select which field of instruction is used to indicate *register number* to be written
- Using this information, we can add instruction labels and extra multiplexor (for *Write register number* input of register file) to datapath

Datapath and Control

- Figure shows these additions plus ALU control block, write signals for state elements, read signal for data memory, and control signals for multiplexors
- Since all multiplexors have two inputs, they each require single control line

Datapath and Control



Control Signals

- Figure shows seven single-bit control lines plus 2-bit *ALUOp* control signal
- Next table describes function of these seven control lines

Effect of Control Signals

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register destination number for the Write register comes from the rt field (bits 20:16).	The register destination number for the Write register comes from the rd field (bits 15:11).
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

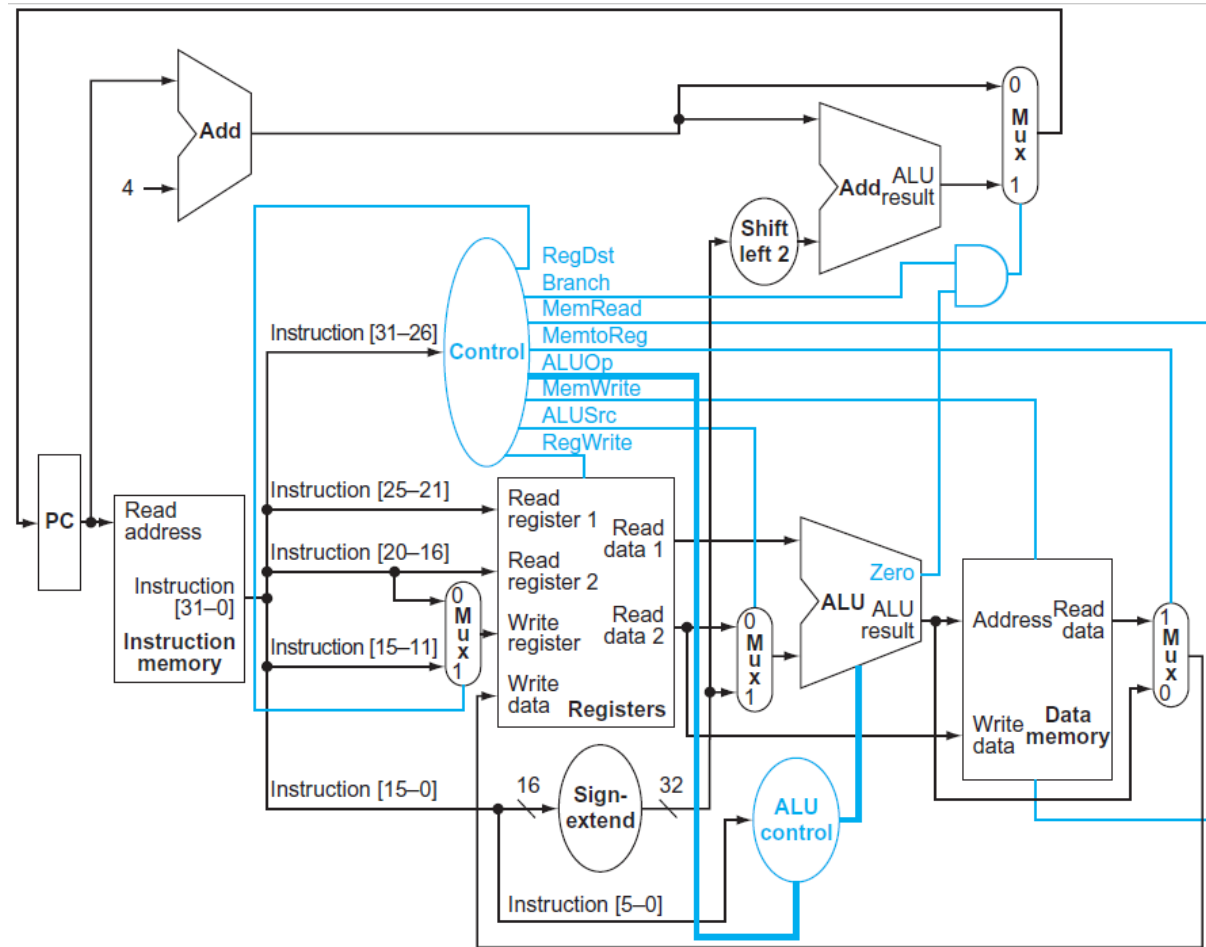
Setting Control Signals

- Control unit can set all but one of control signals based solely on *opcode* field of instruction
- **PCSrc** control line is exception
 - Should be **asserted** if instruction is **beq** (decision that control unit can make) *and* **Zero** output of ALU, which is used for equality comparison, is **asserted**
 - To generate **PCSrc** signal
 - Need to **AND** together signal from control unit, called **Branch**, with **Zero** signal out of ALU

Setting Control Signals

- 9 control signals (7 and two *ALUOp*) can now be set on basis of 6 input signals to control unit, which are **opcode** bits 31 to 26
- Figure shows datapath with control unit and control signals

Datapath with Control Unit



Setting Control Signals by Opcode

- It will be useful to try to define control function informally
- Because setting of control lines depends only on **opcode**, we define whether each control signal should be 0, 1, or don't care (X) for each of **opcode** values
- Figure defines how control signals should be set for each opcode

Setting Control Signals by Opcode

Instruction	RegDst	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1